TEAM PROJECT

AI CULTURAL ADVISOR:

Intelligent Cultural Advisory System

Submitted by

I MCA

NIRANJANA B – 36825007

Under the guidance of

Dr. P. L. CHITHRA

Department of Computer Science

UNIVERSITY OF MADRAS

GUINDY CAMPUS, CHENNAI- 600 025

DEPARTMENT OF COMPUTER SCIENCE

BONAFIDE CERTIFICATE

This is to certify that the record of team project work entitled "CULTURAL ADVISOR" has been successfully completed by our team during the period of JANUARY 2026 – APRIL 2026

Date:

STAFF IN CHARGE	**HEAD OF THE DEPARTMENT**
Dr. P. L. CHITHRA	**Dr. P. L. CHITHRA**

ACKNOWLEDGEMENT

We take this opportunity to acknowledge with great pleasure, deep satisfaction and gratitude, for the contribution of our work in the successful completion of the project.

We express our sincere gratitude and respect to our project guide, Dr. P. L. CHITHRA (Head of the Department), for the constant encouragement and for guiding us until the completion of our project work by providing all the necessary information.

CONTENTS

1	**Introduction** **Problem Definition** **System Objective / Solution**	
2.	**System Study** **Features / Use cases** **Flow Diagram / Business Logic** **Description of Data**	
3.	**System Design** **Project Architecture / AI Model Description** **Project High level design** **Tools and technology stack** **Software Libraries / Components** **Database Design**	
4.	**System Implementation** **Project code structure** **Data Preparation** **Model Training**	
5.	**System Testing**	
6.	**Output**	
7.	**Conclusion**	
8.	**References**	

CULTURAL ADVISOR:

Intelligent Cultural Advisory System

1. INTRODUCTION

1.1 PROBLEM DEFINITION

In the modern era of globalization, people frequently interact with individuals from diverse cultural backgrounds for purposes such as travel, education, business, and social communication. While globalization has made the world more connected, it has also increased the chances of cultural misunderstandings. Each country and region follows its own customs, traditions, etiquette rules, and social expectations, which may not be familiar to outsiders.

For instance, a simple gesture such as greeting someone, dining at a restaurant, or choosing appropriate attire can vary significantly across cultures. A behavior considered polite in one country may be perceived as rude or inappropriate in another. Such misunderstandings can lead to embarrassment, communication barriers, strained relationships, or even professional setbacks.

At present, most cultural information is scattered across various sources such as websites, blogs, and travel guides. These sources are often unstructured, time-consuming to navigate, and lack personalization. Users may find it difficult to obtain precise answers to specific queries like *"What should I wear to a wedding in China?"* or *"How should I greet someone in Japan?"* in a quick and interactive manner.

Therefore, there is a need for an intelligent system that can provide instant, accurate, and context-aware cultural guidance in an easy-to-use conversational format.

1.2 SYSTEM OBJECTIVE / PROPOSED SOLUTION

The AI Cultural Etiquette Advisor System is developed to address the above challenges by providing an intelligent and interactive platform for cultural awareness. The system integrates Artificial Intelligence (AI), Natural Language Processing (NLP), and a structured database to deliver accurate and context-aware responses.

The primary objective of this system is to assist users in understanding cultural norms and etiquette of different countries through a chatbot-based interface. Instead of manually searching for information, users can simply enter their queries in natural language, and the system responds with relevant and meaningful answers.

The system utilizes Natural Language Processing (NLP) techniques to analyze user input, identify user intent, and extract key entities such as country or region names. Based on this analysis, the system retrieves appropriate information from a structured cultural knowledge base stored in the database. In addition, the system integrates with external AI APIs to generate intelligent and conversational responses, ensuring flexibility and completeness of information.

The system also incorporates context awareness, allowing it to remember previous interactions within a session. This enables the chatbot to handle follow-up questions effectively and maintain a natural flow of conversation.

Overall, the system aims to provide:

- Quick access to cultural information
- Personalized and context-aware responses
- A user-friendly conversational experience

1.3 IMPORTANCE OF CULTURAL AWARENESS

Cultural awareness is an essential skill in today's interconnected world. Understanding and respecting cultural differences not only improves communication but also helps in building strong and meaningful relationships.

The importance of cultural awareness can be observed in several areas:

- International Business:
  Professionals working in global environments must be aware of cultural norms related to communication, negotiation styles, dress codes, and business etiquette. This helps in building trust and avoiding misunderstandings during meetings and collaborations.
- Tourism and Travel:
  Travelers visiting foreign countries encounter different traditions and customs. Awareness of local etiquette, such as dining habits, greeting styles, and dress expectations, helps them behave respectfully and enhances their overall travel experience.
- Education and Student Mobility:
  Students studying abroad often face cultural differences in academic and social settings. Understanding these differences enables them to adapt quickly and interact confidently.
- Social and Interpersonal Communication:
  Cultural awareness promotes empathy, respect, and effective communication among individuals from diverse backgrounds, thereby reducing conflicts and misunderstandings.

By addressing these needs, the proposed system serves as a valuable tool for individuals who wish to navigate different cultures with confidence and respect.

1.4 OVERVIEW OF THE SYSTEM

The AI Cultural Etiquette Advisor System is designed as a web-based chatbot application that combines NLP techniques, a structured database, and AI-based response generation to simulate intelligent interaction.

The system allows users to ask questions related to various cultural aspects such as:

- Greetings and communication styles
- Dining etiquette and food habits
- Dress codes for different occasions

- Festivals and traditions
- Gift-giving customs
- Social do’s and don’ts

When a user submits a query, the system processes it through multiple stages, including intent detection, location extraction, and data retrieval. The response is then generated in a conversational and user-friendly format, making it easy for users to understand and apply the information.

The system also includes advanced features such as:

- Context Awareness: Remembers previous queries for better interaction
- Cultural Comparison: Enables comparison between different countries
- Recommendation System: Suggests culturally rich destinations
- API Integration: Fetches additional information when database data is insufficient

By integrating these features, the system not only provides accurate cultural information but also enhances the overall user experience through intelligent and dynamic responses.

1.5 SCOPE OF THE PROJECT

The scope of this project includes the design and development of a chatbot system capable of handling cultural queries related to multiple countries and regions. The system focuses on providing structured and reliable information based on predefined data, NLP techniques, and AI-generated responses.

The current implementation supports text-based interaction through a web interface and maintains user-specific chat sessions for improved usability.

While the system effectively addresses core requirements, it can be further extended in the future to include:

- Advanced machine learning models for improved accuracy
- Multilingual support for global accessibility
- Voice-based interaction for enhanced user experience
- Integration with mobile applications

1.6 SUMMARY

In summary, the AI Cultural Etiquette Advisor System aims to simplify access to cultural knowledge by combining Natural Language Processing, a structured database, and AI-powered conversational interaction. It addresses the challenges of scattered information and lack of personalization by providing a centralized, efficient, and user-friendly solution.

The system enhances cross-cultural understanding by delivering accurate, context-aware, and real-time guidance, making it a valuable tool for travelers, professionals, students, and anyone interacting in a global environment.

2. SYSTEM STUDY

Commented [HB1]:

The system study phase focuses on understanding the existing system, identifying its limitations, and defining the requirements for the proposed system. The current system lacks intelligent guidance and personalization in delivering culturally appropriate recommendations, which leads to inefficiencies and reduced user satisfaction.

The proposed system aims to develop an AI-powered platform that provides culturally sensitive recommendations based on user inputs. It leverages modern technologies such as machine learning models and rule-based logic to enhance decision-making and user experience.

The system is designed to be user-friendly, scalable, and efficient. It integrates backend processing with a responsive frontend interface to ensure seamless interaction. The study also includes feasibility analysis, ensuring that the system is technically, economically, and operationally viable.

2.1 FEATURES / USE CASES

Key Features:

- **User Input Processing:**
  Accepts user queries or inputs related to preferences, context, or requirements.
- **AI-Based Recommendation Engine:**
  Generates intelligent and culturally appropriate suggestions using AI models and predefined rules.
- **Dynamic Response Generation:**
  Produces real-time responses based on user input and system logic.
- **Database Integration:**
  Stores and retrieves user data, rules, and recommendations efficiently.
- **User-Friendly Interface:**
  Provides an intuitive UI for smooth interaction.
- **Error Handling & Validation:**
  Ensures valid inputs and handles system errors gracefully.

Use Cases:

1. **User Interaction:**
   User enters input/query into the system.
2. **Data Processing:**
   System processes input using backend logic.
3. **Recommendation Generation:**
   AI engine generates suitable output.

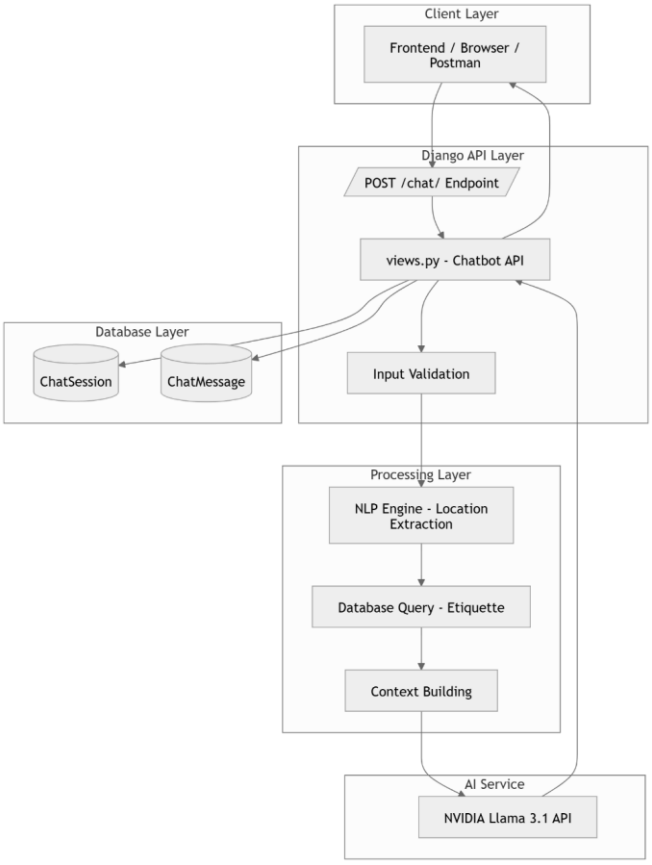
4. **Result Display:**
   Output is shown to the user in a readable format.
5. **Admin Management (Optional):**
   Admin can update rules, datasets, or system configurations.

2.2 FLOW DIAGRAM / BUSINESS LOGIC

Business Logic Explanation:

1. **Input Stage:**
   The user provides input through the interface.
2. **Validation Stage:**
   The system validates the input for correctness and completeness.
3. **Processing Stage:**
   The validated input is sent to the backend where:
   - Rule-based filtering is applied
   - AI model processes the data
4. **Decision Stage:**
   The system determines the most appropriate response based on logic and model output.
5. **Output Stage:**
   The final result is generated and displayed to the user.
6. **Storage Stage:**
   Relevant data may be stored for future reference or improvement

Flow representation:



2.3 DESCRIPTION OF DATA

The system handles various types of data to ensure accurate and efficient functioning.

Types of Data:

- **User Data:**
  Includes user inputs, preferences, and interaction history.
- **System Data:**
  Contains predefined rules, cultural guidelines, and configurations.
- **Training Data (if AI used):**
  Dataset used to train the AI model for generating intelligent responses.
- **Output Data:**
  Generated recommendations or results provided to the user.

Data Storage:

- Data is stored in a structured database (e.g., MySQL).
- Tables may include:
  - User Information
  - Input Logs
  - Recommendation Results
  - Rules/Knowledge Base

Data Handling:

- Data is validated before processing.
- Sensitive data is handled securely.
- Efficient queries are used for fast retrieval.

3. SYSTEM DESIGN

The System Design phase defines the overall architecture, components, modules, and data flow of the Cultural Advisor AI system. It translates the system requirements into a structured blueprint for implementation.

The system is designed using a modular approach to ensure scalability, maintainability, and efficient performance. It integrates web technologies with artificial intelligence to provide culturally appropriate recommendations through an interactive interface.

3.1 PROJECT ARCHITECTURE / AI MODEL DESCRIPTION

Overall Architecture

The Cultural Advisor AI is developed as a Django-based web application following the **Model-View-Template (MVT)** architectural pattern. The system is enhanced with API-based AI integration for dynamic conversational capabilities.

System Components

The system consists of the following layers:

1. **Web Layer**
   Handles HTTP requests and responses using Django views.
2. **Application Layer**
   Contains business logic, NLP processing, and AI integration.
3. **Data Layer**
   Uses SQLite database for storing cultural data and user interactions.
4. **AI Layer**
   Integrates external AI services for natural language response generation.
5. **Frontend Layer**
   Provides user interface using HTML, CSS, and JavaScript.

Architecture Flow

User Interface (Frontend)
↓
Django Backend (Views & Logic)
↓
NLP Processing + Database Query
↓
AI API (Llama Model)
↓
Response Generation
↓
Display to User

AI Model Description

The system integrates **NVIDIA Llama 3.1 70B Instruct Model** through API services.

Model Capabilities

- Natural Language Understanding
- Context-aware response generation
- Cultural knowledge synthesis
- Professional and polite communication

Integration Approach

- REST API calls to external AI service
- Retry mechanism with exponential backoff
- Rate limiting and error handling
- Logging and monitoring of API usage

Prompt Engineering

- Predefined system instructions for cultural advisor role
- Context injection from database
- Controlled response format and length
- Temperature tuning for consistency

NLP Engine

A lightweight NLP module is used to enhance input understanding:

- Intent detection from user queries
- Location extraction using NLP techniques

- Keyword-based fallback mechanism
- Context building for AI processing

3.2 PROJECT HIGH LEVEL DESIGN

Application Flow

User Request
→ URL Routing (Django)
→ View Processing
→ Authentication Check
→ NLP Processing
→ Database Retrieval
→ AI API Call
→ Response Generation
→ Data Storage
→ Response to User Interface

Main Modules

1. **User Interface Module**
   Handles user interaction and displays results.
2. **Input Processing Module**
   Validates and preprocesses user input.
3. **Business Logic Module**
   Controls system workflow and applies rules.
4. **AI Processing Module**
   Generates intelligent and context-aware responses.
5. **Database Module**
   Stores and retrieves system data.

Design Patterns Used

- MVT Architecture (Django)
- Service Layer for AI and NLP integration
- Repository Pattern for database handling
- Strategy Pattern for extensibility of AI providers

Security Design

- User authentication using Django Auth
- CSRF protection
- Secure API key management using environment variables

- Input validation and sanitization

Scalability Considerations

- Stateless API design
- Database indexing for faster queries
- Caching of API responses
- Support for asynchronous processing

3.3 TOOLS AND TECHNOLOGY STACK

Backend Technologies

- Python 3.x
- Django 5.1.4

Frontend Technologies

- HTML
- CSS
- JavaScript

Database

- SQLite 3 (lightweight and efficient for development)

AI Technologies

- NVIDIA AI API (Llama 3.1 70B Model)

Development Tools

- Visual Studio Code
- Git (Version Control)
- PowerShell / Command Line

Deployment Tools

- Gunicorn (WSGI Server)
- Nginx (Web Server)
- Docker (optional containerization)

3.4 SOFTWARE LIBRARIES / COMPONENTS

Core Libraries

- Django – Web framework

- asgiref – ASGI utilities
- sqlparse – SQL parsing

AI and NLP Libraries

- requests – API communication
- spaCy – Natural Language Processing

Web Development Libraries

- python-dotenv – Environment variable management
- python-decouple – Configuration management
- beautifulsoup4 – HTML parsing

Development Tools

- **Django** – Web framework used for backend development and API handling
- **Antigravity** – Used for lightweight development environment and project setup
- **Git Bash** – Command-line tool used for version control and project management

3.5 DATABASE DESIGN

Database Overview

The system uses **SQLite relational database** to store cultural data, user interactions, and system logs. The design ensures data consistency, scalability, and efficient retrieval.

Core Tables

1. ChatSession Table

Manages chat sessions for each user.

Fields include:

- User (Foreign Key)
- Session title
- Created and updated timestamps
- Last accessed country
- Flag display status

2. ChatMessage Table

Stores messages exchanged in each session.

Fields include:

- Session (Foreign Key)
- Message content
- Sender (User/Bot)
- Timestamp

3. APILog Table

Tracks API usage and performance.

Fields include:

- API name
- Endpoint
- Query data
- Success status
- Latency
- Timestamp

Relationships

- One User → Many ChatSessions
- One ChatSession → Many ChatMessages

Database Design Principles

- Normalization (3NF)
- Efficient indexing
- Data integrity using foreign keys
- Flexible schema for cultural data

Data Flow

1. User input is received
2. NLP extracts relevant information
3. Database is queried for cultural data
4. Data is passed to AI model
5. Response is generated and stored
6. Output is displayed to the user

Performance Optimization

- Indexed fields for faster lookup
- Optimized queries
- Efficient storage of large text data
- Logging for monitoring system performance

4. SYSTEM IMPLEMENTATION

The System Implementation phase describes how the designed system is developed, structured, and executed in practice. The Cultural Advisor AI is implemented as a Django-based web application integrating database management, NLP processing, and external AI services to deliver intelligent cultural recommendations.

4.1 PROJECT CODE STRUCTURE

Project Layout

The project follows a modular structure based on Django framework conventions.

- **cultural_advisor/**
  - manage.py — Entry point for Django commands
  - **cultural_advisor/**
    - settings.py — Project configuration settings
    - urls.py — Root URL routing
    - wsgi.py — WSGI application interface
    - asgi.py — ASGI application interface
- **advisor/** (Main Application)
  - apps.py — Application configuration
  - models.py — Database models definition
  - views.py — Backend logic and API endpoints
  - urls.py — Application-level routing
  - nlp_engine.py — NLP and location extraction logic
  - gemini_service.py — AI API integration
  - admin.py — Admin panel configuration
  - tests.py — Unit testing placeholder
  - migrations/ — Database migration files
- **Frontend Files**
  - templates/advisor/ — HTML templates
  - static/advisor/js/ — JavaScript files
  - static/advisor/css/ — Styling files

Key Implementation Files

advisor/views.py

- Contains main application logic and API endpoints
- Handles user requests and responses
- Key functions:
  - chatbot() — Processes chat requests, calls AI API, stores responses
  - home() — Loads main interface
  - signup_view(), login_view(), logout_view() — Authentication
  - get_chat_history() — Retrieves past sessions
  - get_session_messages() — Fetches messages
  - delete_chat_session() — Deletes session data

advisor/models.py

Defines database schema:

- Etiquette — Cultural knowledge base
- ChatSession — User session tracking
- ChatMessage — Chat message storage
- APILog — API monitoring and logging

advisor/nlp_engine.py

- Extracts location from user input
- Uses spaCy (if available)
- Includes fallback keyword matching
- Builds context for AI processing

advisor/gemini_service.py

- Handles communication with NVIDIA AI API
- Uses requests library
- Implements retry logic and error handling
- Sends prompts to **Llama 3.1 70B model**
- Processes AI responses

Frontend Integration

- index.html — Main chatbot interface
- login.html — User login page
- signup.html — User registration page
- chat.js — Handles chat interactions
- style.css, index_css.css — UI design

4.2 DATA PREPARATION

Data Sources and Storage

- Primary database: SQLite (db.sqlite3)

Data Model Details

Tables

- ChatSession — Stores user sessions
- ChatMessage — Stores conversation history
- APILog — Tracks API usage

Runtime Data Flow

1. User sends message
2. Request handled by views.chatbot()
3. NLP extracts location
4. Database retrieves cultural data
5. Context is added to prompt
6. AI API generates response
7. Data stored in database
8. Response displayed to user

Environment Configuration

- .env file stores sensitive data:
    - NVIDIA_API_KEY
    - Database credentials

- Managed using:
    - python-dotenv
    - python-decouple

4.3 MODEL TRAINING

Actual Model Usage

- No local model training is performed
- System uses **pre-trained external AI model**
- AI accessed via REST API

Implementation Approach

Prompt Engineering

- Structured prompts created dynamically
- Includes:
    - System instructions
    - Cultural context from database
    - User query
- Ensures:
    - Clear, polite responses
    - Cultural relevance
    - No unnecessary data exposure

Error Handling

- Retry mechanism for failed API calls
- Handles:
    - Timeouts
    - API failures
    - Invalid responses
- Provides fallback responses

Logging and Monitoring

- API calls logged in APILog table
- Tracks:
  - Success/failure
  - Response time (latency)

Important Note on Training

- No machine learning model is trained locally
- The system uses **external LLM inference only**
- Data preparation refers to:
  - Building and maintaining cultural database
  - Not training model weights

4.4 SUMMARY

The Cultural Advisor AI system is implemented as a full-stack Django application with:

- Modular project structure
- Integrated cultural knowledge database
- NLP-based input processing
- AI-powered response generation using external LLM
- Secure and scalable architecture

The system successfully combines **web development and AI technologies** to provide intelligent, culturally aware recommendations without requiring local model training.

5. SYSTEM TESTING

System Testing is performed to evaluate the complete and integrated system to ensure it meets the specified requirements. In this project, system testing validates the functionality, performance, and reliability of the Cultural Advisor AI application.

5.1 PURPOSE

The purpose of system testing is to verify that the entire application works correctly as an integrated system.

This includes:

- User authentication (signup, login, logout)
- Chat message processing
- Database storage and retrieval
- AI API integration
- Frontend and backend communication

The goal is to ensure that all components interact seamlessly and provide accurate outputs.

5.2 TEST SCOPE

The testing process covers the following functionalities:

- User registration, login, and logout
- Access control for authenticated users
- Chat request handling (POST /chat/)
- Chat history and session retrieval
- Chat session deletion
- Error handling for invalid inputs and API failures
- NLP-based location extraction
- Database-driven context enrichment

5.3 TEST TYPES

Functional Testing

Functional testing ensures that each feature of the system works as expected.

- Verify user authentication (signup, login, logout)

- Ensure chat interface loads only for authenticated users
- Validate chatbot API responses for valid requests
- Ensure invalid requests (GET/empty input) are rejected
- Verify chat history and session management

Integration Testing

Integration testing validates the interaction between different modules.

- Request processing in views
- NLP location extraction
- Database lookup (Etiquette table)
- AI API communication
- Storage of chat messages
- Response delivery to frontend

API Testing

API testing checks the correctness of backend endpoints.

- Validate JSON response structure
- Check HTTP status codes
- Ensure authentication is enforced
- Verify session ownership restrictions

Regression Testing

Regression testing ensures that new updates do not break existing functionality.

- Verify fallback when AI API fails
- Ensure chatbot works without location detection
- Validate database consistency after migrations

5.4 TEST ENVIRONMENT

The testing environment includes:

- Django built-in test framework
- SQLite test database (temporary or in-memory)
- Environment variables loaded from .env

Test Commands:

- python manage.py test advisor
- python manage.py test

5.5 TEST IMPLEMENTATION NOTES

- Testing module: advisor/tests.py
- Currently contains placeholder code
- Requires implementation of unit and integration tests

5.6 RECOMMENDED TEST CASES

Authentication Tests

- test_signup_creates_user
- test_login_authenticates_user
- test_logout_redirects_to_login
- test_protected_views_require_login

Chat Endpoint Tests

- test_chatbot_rejects_get
- test_chatbot_rejects_empty_message
- test_chatbot_creates_chat_session_and_message
- test_chatbot_returns_reply_json
- test_chatbot_uses_location_context_when_available

History and Session Tests

- test_get_chat_history_returns_user_sessions
- test_get_session_messages_returns_session_messages
- test_delete_chat_session_removes_session

Error Handling Tests

- test_chatbot_handles_api_failure
- test_get_session_messages_404_for_invalid_session
- test_delete_chat_session_only_deletes_user_session

5.7 EXTERNAL API TESTING

Since the system depends on an external AI API, testing includes simulation techniques:

- Mock API calls using requests.post
- Simulate successful response (HTTP 200)
- Simulate errors (401 Unauthorized, 429 Rate Limit, Timeout)
- Validate fallback responses during failures

This ensures the system behaves reliably even when external services fail.

5.8 MANUAL SYSTEM TESTING

Manual testing is performed to validate real-time behavior:

1. Create a new user account
2. Login to the system
3. Access the chatbot interface
4. Send queries related to countries or regions
5. Verify AI-generated responses
6. Check chat history functionality
7. Delete chat sessions and verify removal
8. Logout and test unauthorized access

5.9 ACCEPTANCE CRITERIA

The system is considered successful if:

- Authenticated users can access and use the chatbot
- Chat messages are stored and retrieved correctly
- Cultural context is applied in responses
- Invalid requests are handled properly
- External API failures do not crash the system

6. OUTPUT

The Output section presents the results generated by the Cultural Advisor AI system. It demonstrates the working of the application through various user interactions and system responses.

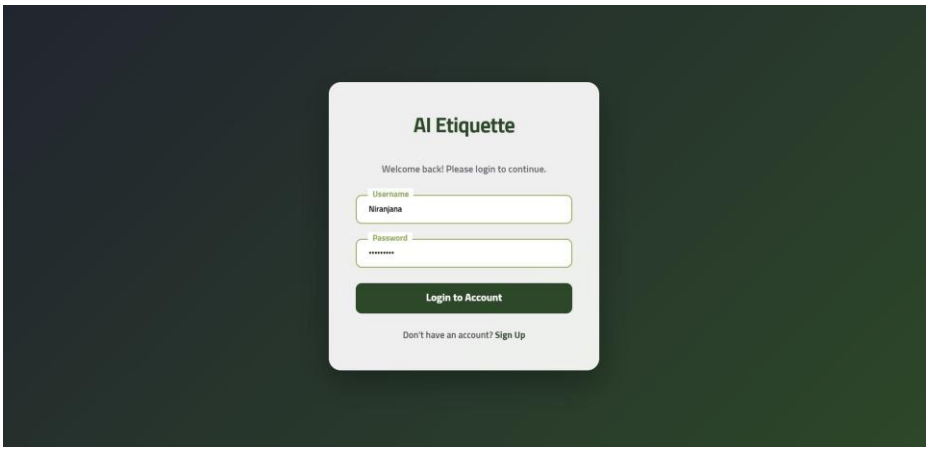
The outputs validate that the system successfully processes user input, integrates cultural data, and generates intelligent responses using AI.

6.1 USER INTERFACE OUTPUT

This section includes screenshots of the frontend interface.

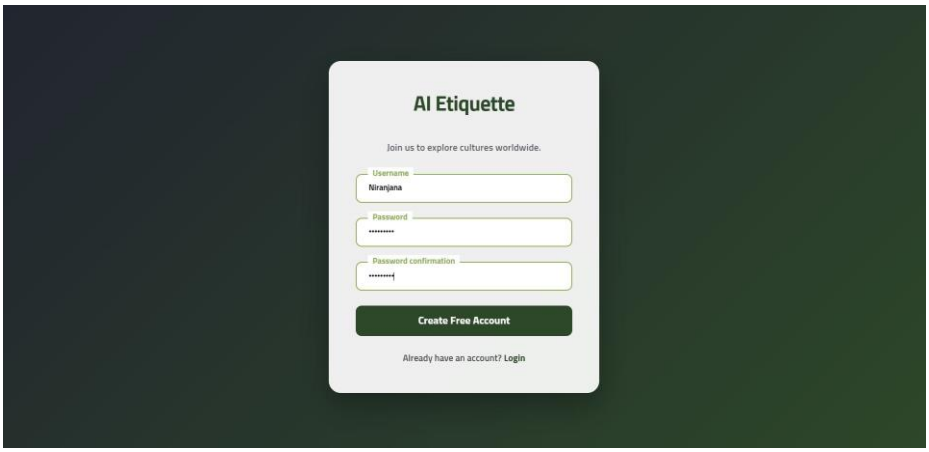
Login Page

- Displays fields for username and password
- Allows registered users to log in



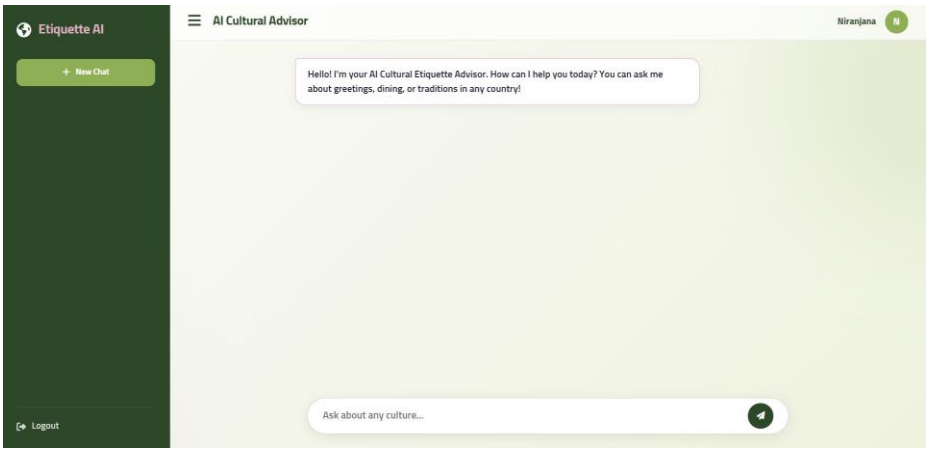
Signup Page

- Allows new users to register
- Collects user credentials



Chat Interface

- Main interface of the application
- Users can type queries and view responses



6.2 CHATBOT RESPONSE OUTPUT

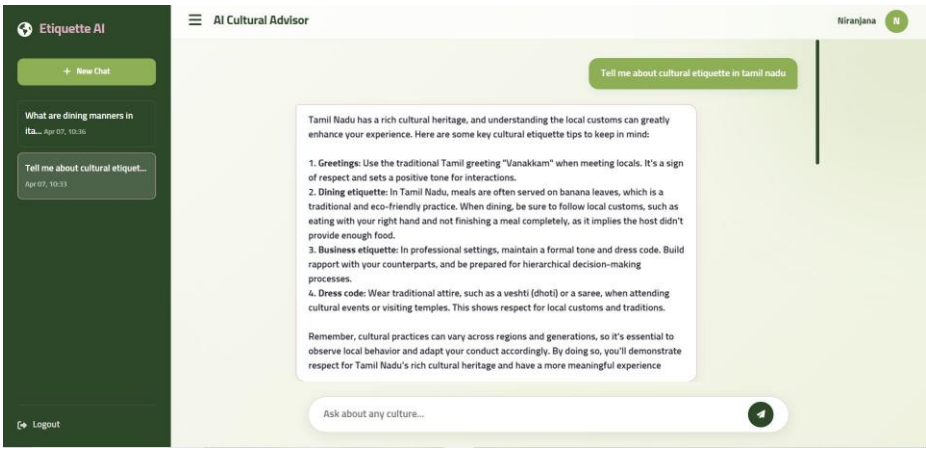
This section shows how the system responds to user queries.

Sample Input 1:

"Tell me about cultural etiquette in Tamil Nadu"

Output:

- Traditional greeting style (“Vanakkam”)
- Dining etiquette (banana leaf serving, eating with right hand)
- Business etiquette (formal behavior and hierarchy)
- Traditional dress code (veshti, saree for cultural occasions)
- Importance of respecting local customs and traditions

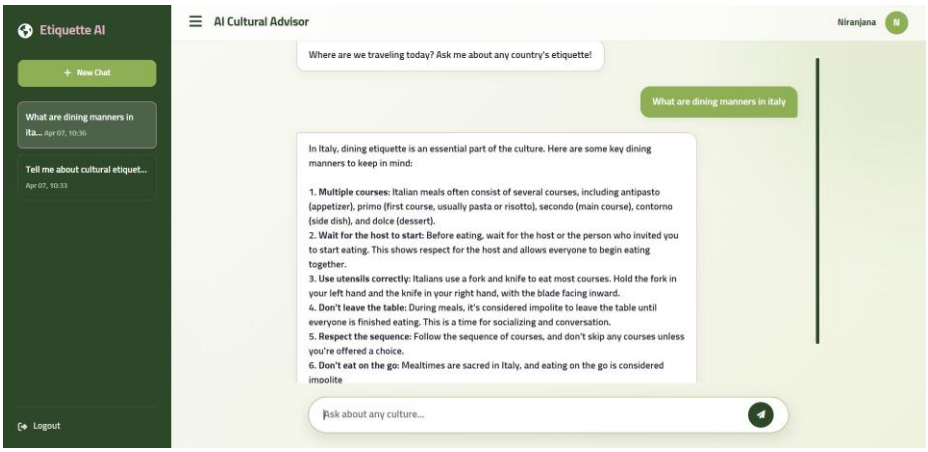


Sample Input 2:

“What are dining manners in Italy?”

Output:

- Multiple course meal structure (antipasto, primo, secondo, etc.)
- Waiting for the host to start the meal
- Proper use of utensils (fork and knife)
- Importance of staying at the table until everyone finishes
- Following the sequence of courses
- Avoiding eating on the go



6.3 DATABASE OUTPUT

This section demonstrates how data is stored in the database.

ChatSession Table

- Stores user session details

ChatMessage Table

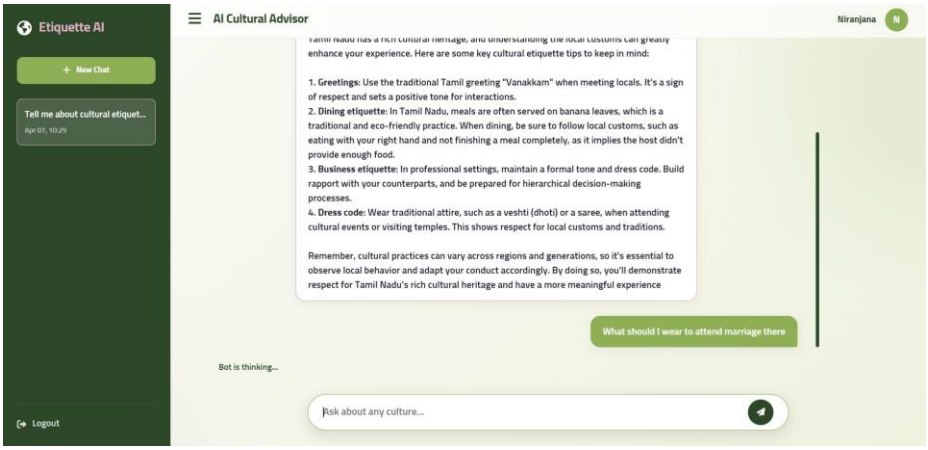
- Stores user and bot messages

id	title	created_at	updated_at	user_id	flag_sent	last_country
1	Tell me the cultural etiquette...	2026-03-31 14:58:54.391057	2026-03-31 15:03:51.627060	1	0	NULL
2	hi	2026-04-01 09:32:01.724135	2026-04-03 08:43:44.893442	2	0	NULL
3	hello	2026-04-03 08:44:53.430438	2026-04-03 09:38:05.345544	2	0	NULL
4	tell me about japan's cultural...	2026-04-03 09:39:25.382475	2026-04-03 10:29:40.362927	2	0	NULL
5	dining tips of japan...	2026-04-03 10:40:53.539327	2026-04-03 10:40:53.539327	1	0	NULL
6	can you give me cultural etiqu...	2026-04-03 10:51:31.707882	2026-04-03 10:55:48.060930	2	0	NULL
7	hello, tell me tips to visit i...	2026-04-03 11:20:55.974509	2026-04-03 11:20:58.422442	2	0	NULL
8	tell me the tips to visit indi...	2026-04-03 11:25:10.711120	2026-04-03 11:25:47.721992	2	0	NULL
9	Give me tips to wear on india...	2026-04-03 12:10:04.325176	2026-04-03 12:10:06.566327	3	0	NULL
10	Give me tips about what to wea...	2026-04-03 12:27:08.156473	2026-04-03 12:27:15.771732	3	0	NULL
11	what to wear on japan...	2026-04-05 10:33:55.953345	2026-04-05 10:33:57.387784	4	0	NULL
12	india...	2026-04-05 10:34:13.461090	2026-04-05 10:34:13.461090	4	0	NULL
13	india...	2026-04-05 10:34:32.522852	2026-04-05 10:34:36.281090	4	0	NULL
14	what to wear to attend marriag...	2026-04-05 15:08:47.969891	2026-04-05 15:08:47.970687	4	0	NULL
15	cultural etiquette about india...	2026-04-05 15:09:07.332233	2026-04-05 15:09:07.332233	4	0	NULL
16	cultural etiquette about japan...	2026-04-05 15:19:46.529703	2026-04-05 15:19:46.529703	4	0	NULL
17	cultural etiquette about japan...	2026-04-05 15:34:03.196567	2026-04-05 15:34:03.196567	4	0	NULL
18	cultural etiquette about japan...	2026-04-05 16:28:10.190226	2026-04-05 16:28:10.190226	4	0	NULL
19	hii...	2026-04-05 16:35:49.882406	2026-04-05 16:35:49.882406	4	0	NULL
20	tell me about japan...	2026-04-05 16:36:13.061398	2026-04-05 16:36:13.061398	4	0	NULL
21	cultural etiquette about japan...	2026-04-05 16:36:23.229196	2026-04-05 16:36:23.229196	4	0	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.4 FUNCTIONAL OUTPUT

This section shows system functionalities:

- Successful user authentication
- Chat message processing
- Real-time AI response generation
- Chat history retrieval
- Session deletion

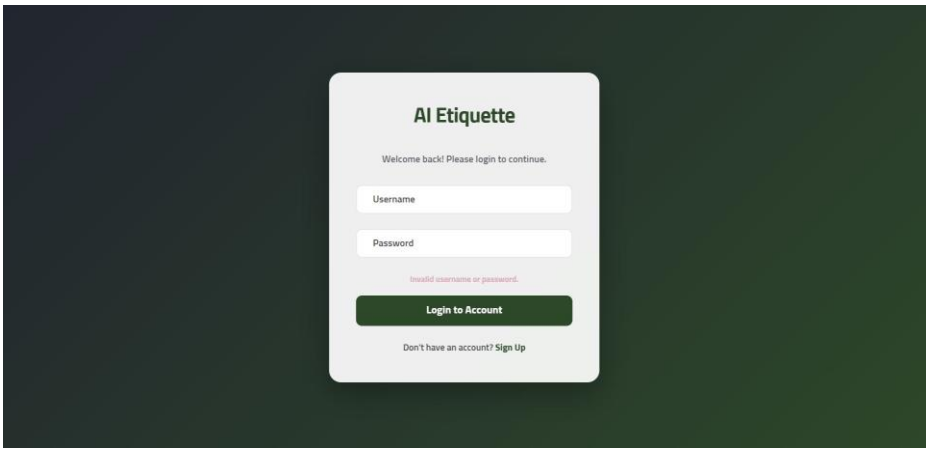


6.5 ERROR HANDLING OUTPUT

This section demonstrates how the system handles errors.

Examples:

- Empty input → Error message displayed
- API failure → Fallback response shown
- Unauthorized access → Redirect to login



6.7 SUMMARY

The outputs confirm that the system:

- Successfully integrates frontend, backend, and AI components
- Provides accurate and culturally relevant responses
- Maintains user sessions and chat history
- Handles errors effectively

7. CONCLUSION

The Cultural Advisor AI system has been successfully designed and implemented as an intelligent web-based application that provides culturally relevant guidance to users. The system effectively integrates web technologies, natural language processing, and artificial intelligence to deliver meaningful and context-aware responses.

The application achieves its primary objective of assisting users in understanding cultural etiquettes across different regions. By combining a structured cultural knowledge database with an external AI model, the system ensures both accuracy and flexibility in response generation.

The use of Django framework enabled efficient backend development, while the integration of NLP techniques helped in identifying user intent and extracting location-based information. The external AI model further enhanced the system by generating natural, human-like responses.

The system also ensures proper data management by storing user interactions, maintaining chat history, and handling API communications effectively. Additionally, error handling mechanisms and fallback responses improve system reliability and user experience.

Overall, the project demonstrates the practical implementation of AI in real-world applications, particularly in enhancing cultural awareness and user interaction. The system is scalable, user-friendly, and capable of further improvements, making it a strong foundation for future development.

8. REFERENCES

[1] Django Software Foundation, "Django Web Framework Documentation," 2024.

[2] Python Software Foundation, "Python 3 Documentation," 2024.

[3] NVIDIA Corporation, "NVIDIA NIM API Documentation for Llama 3.1 Models," 2024.

[4] Meta AI, "Llama 3.1 Large Language Model Overview," 2024.

[5] Explosion AI, "spaCy Natural Language Processing Library Documentation," 2024.

[6] SQLite Consortium, "SQLite Database Documentation," 2024.

[7] Reitz, K., "Requests: HTTP for Humans – Python Library Documentation," 2024.